第16章 数 组

在第5章的末尾，你学习了如何定义并初始化一个数组。

对数组的基本看法是，你可以创建并组装它们，通过使用整型索引值访问它们的元素，并且它们的尺寸不能改变。在大多数时候，这就是你需要了解的全部，但是有时你需要在数组上执行更加复杂的操作，并且你可能需要评估到底是使用数组还是更加灵活的容器。本章将向你展示如何更加深入地思考数组。

16.1 数组为什么特殊

Java中有大量其他的方式可以持有对象，那么，到底是什么使数组变得与众不同呢？

数组与其他种类的容器之间的区别有三方面：效率、类型和保存基本类型的能力。在Java中，数组是一种效率最高的存储和随机访问对象引用序列的方式。数组就是一个简单的线性序列，这使得元素访问非常快速。但是为这种速度所付出的代价是数组对象的大小被固定，并且在其生命周期中不可改变。你可能会建议使用**ArrayList**（参见第11章），它可以通过创建一个新实例，然后把旧实例中所有的引用移到新实例中，从而实现更多空间的自动分配。尽管通常应该首选**ArrayList**而不是数组，但是这种弹性需要开销，因此，**ArrayList**的效率比数组低很多。

数组和容器都可以保证你不能滥用它们。无论你是使用数组还是容器，如果越界，都会得到一个表示程序员错误的**RuntimeException**异常。

在泛型之前，其他的容器类在处理对象时，都将它们视作没有任何具体类型。也就是说，它们将这些对象都当作Java中所有类的根类**Object**处理。数组之所以优于泛型之前的容器，就是因为你可以创建一个数组去持有某种具体类型。这意味着你可以通过编译期检查，来防止插入错误类型和抽取不当类型。当然，无论在编译时还是运行时，Java都会阻止你向对象发送不恰当的消息。所以，并不是说哪种方法更不安全，只是如果编译时就能够指出错误，会显得更加优雅，也减少了程序的使用者被异常吓着的可能性。

数组可以持有基本类型，而泛型之前的容器则不能。但是有了泛型，容器就可以指定并检查它们所持有对象的类型，并且有了自动包装机制，容器看起来还能够持有基本类型。下面是将数组与泛型容器进行比较的示例：

```
//: arrays/ContainerComparison.java
import java.util.*;
import static net.mindview.util.Print.*;

class BerylliumSphere {
  private static long counter;
  private final long id = counter++;
  public String toString() { return "Sphere " + id; }
}

public class ContainerComparison {
  public static void main(String[] args) {
    BerylliumSphere[] spheres = new BerylliumSphere[10];
    for(int i = 0; i < 5; i++)
      spheres[i] = new BerylliumSphere();
    print(Arrays.toString(spheres));
    print(spheres[4]);
```

```
    List<BerylliumSphere> sphereList =
      new ArrayList<BerylliumSphere>();
    for(int i = 0; i < 5; i++)
      sphereList.add(new BerylliumSphere());
    print(sphereList);
    print(sphereList.get(4));

    int[] integers = { 0, 1, 2, 3, 4, 5 };
    print(Arrays.toString(integers));
    print(integers[4]);

    List<Integer> intList = new ArrayList<Integer>(
      Arrays.asList(0, 1, 2, 3, 4, 5));
    intList.add(97);
    print(intList);
    print(intList.get(4));
  }
} /* Output:
[Sphere 0, Sphere 1, Sphere 2, Sphere 3, Sphere 4, null,
null, null, null, null]
Sphere 4
[Sphere 5, Sphere 6, Sphere 7, Sphere 8, Sphere 9]
Sphere 9
[0, 1, 2, 3, 4, 5]
4
[0, 1, 2, 3, 4, 5, 97]
4
*///:~
```

748

这两种持有对象的方式都是类型检查型的，并且唯一明显的差异就是数组使用[]来访问元素，而**List**使用的是**add()**和**get()**这样的方法。数组和**ArrayList**之间的相似性是有意设计的，这使得从概念上讲，这两者之间的切换是很容易的。但是正如你在第11章中所看到的，容器比数组明显具有更多的功能。

随着自动包装机制的出现，容器已经可以与数组几乎一样方便地用于基本类型中了。数组硕果仅存的优点就是效率。然而，如果要解决更一般化的问题，那数组就可能会受到过多的限制，因此在这些情形下你还是会使用容器。

16.2 数组是第一级对象

无论使用哪种类型的数组，数组标识符其实只是一个引用，指向在堆中创建的一个真实对象，这个（数组）对象用以保存指向其他对象的引用。可以作为数组初始化语法的一部分隐式地创建此对象，或者用**new**表达式显式地创建。只读成员**length** 是数组对象的一部分（事实上，这是唯一一个可以访问的字段或方法），表示此数组对象可以存储多少元素。“[]”语法是访问数组对象唯一的方式。

下例总结了初始化数组的各种方式，以及如何对指向数组的引用赋值，使之指向另一个数组对象。此例也说明，对象数组和基本类型数组在使用上几乎是相同的；唯一的区别就是对象数组保存的是引用，基本类型数组直接保存基本类型的值。

749

```
//: arrays/ArrayOptions.java
// Initialization & re-assignment of arrays.
import java.util.*;
import static net.mindview.util.Print.*;

public class ArrayOptions {
  public static void main(String[] args) {
    // Arrays of objects:
    BerylliumSphere[] a; // Local uninitialized variable
    BerylliumSphere[] b = new BerylliumSphere[5];
```

```
    // The references inside the array are
    // automatically initialized to null:
    print("b: " + Arrays.toString(b));
    BerylliumSphere[] c = new BerylliumSphere[4];
    for(int i = 0; i < c.length; i++)
      if(c[i] == null) // Can test for null reference
        c[i] = new BerylliumSphere();
    // Aggregate initialization:
    BerylliumSphere[] d = { new BerylliumSphere(),
      new BerylliumSphere(), new BerylliumSphere()
    };
    // Dynamic aggregate initialization:
    a = new BerylliumSphere[]{
      new BerylliumSphere(), new BerylliumSphere(),
    };
    // (Trailing comma is optional in both cases)
    print("a.length = " + a.length);
    print("b.length = " + b.length);
    print("c.length = " + c.length);
    print("d.length = " + d.length);
    a = d;
    print("a.length = " + a.length);

    // Arrays of primitives:
    int[] e; // Null reference
    int[] f = new int[5];
    // The primitives inside the array are
    // automatically initialized to zero:
    print("f: " + Arrays.toString(f));
    int[] g = new int[4];
    for(int i = 0; i < g.length; i++)
      g[i] = i*i;
    int[] h = { 11, 47, 93 };
    // Compile error: variable e not initialized:
    //!print("e.length = " + e.length);
    print("f.length = " + f.length);
    print("g.length = " + g.length);
    print("h.length = " + h.length);
    e = h;
    print("e.length = " + e.length);
    e = new int[]{ 1, 2 };
    print("e.length = " + e.length);
  }
} /* Output:
b: [null, null, null, null, null]
a.length = 2
b.length = 5
c.length = 4
d.length = 3
a.length = 3
f: [0, 0, 0, 0, 0]
f.length = 5
g.length = 4
h.length = 3
e.length = 3
e.length = 2
*///:~
```

750

数组**a**是一个尚未初始化的局部变量，在你对它正确地初始化之前，编译器不允许用此引用做任何事情。数组**b**初始化为指向一个**BerylliumSphere**引用的数组，但其实并没有**BerylliumSphere**对象置入数组中。然而，仍然可以询问数组的大小，因为**b**指向一个合法的对象。这样做有一个小缺点：你无法知道在此数组中确切地有多少元素，因为**length**只表示数组能够容纳多少元素。也就是说，**length**是数组的大小，而不是实际保存的元素个数。新生成一个数组对象时，其中所有的引用被自动初始化为**null**；所以检查其中的引用是否为**null**，即可知道数组的

某个位置是否存有对象。同样，基本类型的数组如果是数值型的，就被自动初始化为0；如果是字符型（**char**）的，就被自动初始化为**(char)O**；如果是布尔型（**boolean**），就被自动初始化为**false**。

751

数组**c**表明，数组对象在创建之后，随即将数组的各个位置都赋值为**BerylliumSphere**对象。数组**d**表明使用“聚集初始化”语法创建数组对象（隐式地使用**new**在堆中创建，就像数组**c**一样），并且以**BerylliumSphere**对象将其初始化的过程，这些操作只用了一条语句。

下一个数组初始化可以看作是“动态的聚集初始化”。数组**d**采用的聚集初始化操作必须在定义**d**的位置使用，但若使用第二种语法，可以在任意位置创建和初始化数组对象。例如，假设方法**hide()**需要一个**BerylliumSphere**对象的数组作为输入参数。可以如下调用：

```
hide(d);
```

但也可以动态地创建将要作为参数传递的数组：

```
hide(new BerylliumSphere[]{ new BerylliumSphere(),
```

在许多情况下，此语法使得代码书写变得更方便了。

表达式：

```
a = d;
```

说明如何将指向某个数组对象的引用赋给另一个数组对象，这与其他类型的对象引用没什么区别。现在**a**与**d**都指向堆中的同一个数组对象。

ArraySize.java的第二部分说明，基本类型数组的工作方式与对象数组一样，不过基本类型的数组直接存储基本类型数据的值。

练习1：(2) 创建一个受**BerylliumSphere**数组作为参数的方法，并动态地创建参数去调用这个方法。证明在本例中普通的聚集数组初始化不能奏效。去发现总结在哪些情况下，普通的聚集初始化可以起作用，而又在哪些情况下，动态聚集初始化显得多余。

752

16.3 返回一个数组

假设你要写一个方法，而且希望它返回的不止一个值，而是一组值。这对于C和C++这样的语言来说就有点困难，因为它们不能返回一个数组，而只能返回指向数组的指针。这会造成一些问题，因为它使得控制数组的生命周期变得很困难，并且容易造成内存泄漏。

在Java中，你只是直接“返回一个数组”，而无需担心要为数组负责——只要你需要它，它就会一直存在，当你使用完后，垃圾回收器会清理掉它。

下例演示如何返回**String**型数组：

```
//: arrays/IceCream.java
// Returning arrays from methods.
import java.util.*;

public class IceCream {
  private static Random rand = new Random(47);
  static final String[] FLAVORS = {
    "Chocolate", "Strawberry", "Vanilla Fudge Swirl",
    "Mint Chip", "Mocha Almond Fudge", "Rum Raisin",
    "Praline Cream", "Mud Pie"
  };
  public static String[] flavorSet(int n) {
    if(n > FLAVORS.length)
      throw new IllegalArgumentException("Set too big");
    String[] results = new String[n];
    boolean[] picked = new boolean[FLAVORS.length];
    for(int i = 0; i < n; i++) {
```

```
      int t;
      do
        t = rand.nextInt(FLAVORS.length);
      while(picked[t]);
      results[i] = FLAVORS[t];
      picked[t] = true;
    }
    return results;
  }
  public static void main(String[] args) {
    for(int i = 0; i < 7; i++)
      System.out.println(Arrays.toString(flavorSet(3)));
  }
} /* Output:
[Rum Raisin, Mint Chip, Mocha Almond Fudge]
[Chocolate, Strawberry, Mocha Almond Fudge]
[Strawberry, Mint Chip, Mocha Almond Fudge]
[Rum Raisin, Vanilla Fudge Swirl, Mud Pie]
[Vanilla Fudge Swirl, Chocolate, Mocha Almond Fudge]
[Praline Cream, Strawberry, Mocha Almond Fudge]
[Mocha Almond Fudge, Strawberry, Mint Chip]
*///:~
```

753

方法**flavorSet()**创建了一个名为**results**的**String**数组。此数组容量为**n**，由传入方法的参数决定。然后从数组**FLAVORS**中随机选择元素（即“味道”），存入**results**数组中，它是方法所最终返回的数组。返回一个数组与返回任何其他对象（实质上是返回引用）没什么区别。数组是在**flavorSet()**中被创建还是在别的地方被创建，这一点并不重要。当使用完毕后，垃圾回收器负责清理数组；而只要还需要它，此数组就会一直存在。

说句题外话，注意当**flavorSet()**随机选择各种数组元素“味道”时，它确保不会重复选择。由一个**do**循环不断进行随机选择，直到找出一个在数组**picked**中还不存在的元素。（当然，还会比较**String**以检查随机选择的元素是否已经在数组**results**中。）如果成功，将此元素加入数组，然后查找下一个（**i**递增）。

从输出中可以看出，**flavorSet()** 每次确实是在随机选择“味道”。

练习2：(1) 编写一个方法，它接受一个**int**参数，并返回一个具有该尺寸的数组，用**BerylliumSphere**对象填充该数组。

16.4 多维数组

创建多维数组很方便。对于基本类型的多维数组，可以通过使用花括号将每个向量分隔开：

```
//: arrays/MultidimensionalPrimitiveArray.java
// Creating multidimensional arrays.
import java.util.*;

public class MultidimensionalPrimitiveArray {
  public static void main(String[] args) {
    int[][] a = {
      { 1, 2, 3, },
      { 4, 5, 6, },
    };
    System.out.println(Arrays.deepToString(a));
  }
} /* Output:
[[1, 2, 3], [4, 5, 6]]
*///:~
```

754

每对花括号括起来的集合都会把你带到下一级数组。

下面的示例使用了Java SE5的**Arrays.deepToString()**方法，它可以将多维数组转换为多个

String，正如从输出中所看到的那样。还可以使用**new**来分配数组，下面的三维数组就是在**new**表达式中分配的：

```
//: arrays/ThreeDWithNew.java
import java.util.*;

public class ThreeDWithNew {
  public static void main(String[] args) {
    // 3-D array with fixed length:
    int[][][] a = new int[2][2][4];
    System.out.println(Arrays.deepToString(a));
  }
} /* Output:
[[[0, 0, 0, 0], [0, 0, 0, 0]], [[0, 0, 0, 0], [0, 0, 0,
0]]]
*///:~
```

你可以看到基本类型数组的值在不进行显式初始化的情况下，会被自动初始化。对象数组会被初始化为**null**。

数组中构成矩阵的每个向量都可以具有任意的长度（这被称为*粗糙数组*）：

755

```
//: arrays/RaggedArray.java
import java.util.*;

public class RaggedArray {
  public static void main(String[] args) {
    Random rand = new Random(47);
    // 3-D array with varied-length vectors:
    int[][][] a = new int[rand.nextInt(7)][][];
    for(int i = 0; i < a.length; i++) {
      a[i] = new int[rand.nextInt(5)][];
      for(int j = 0; j < a[i].length; j++)
        a[i][j] = new int[rand.nextInt(5)];
    }
    System.out.println(Arrays.deepToString(a));
  }
} /* Output:
[[], [[0], [0], [0, 0, 0, 0]], [[], [0, 0], [0, 0]], [[0,
0, 0], [0], [0, 0, 0, 0]], [[0, 0, 0], [0, 0, 0], [0], []],
[[0], [], [0]]]
*///:~
```

第一个**new**创建了数组，其第一维的长度是由随机数确定的，其他维的长度则没有定义。位于**for**循环内的第二个**new**则会决定第二维的长度；直到碰到第三个**new**，第三维的长度才得以确定。

可以用类似的方式处理非基本类型的对象数组。下面，你可以看到如何用花括号把多个**new**表达式组织到一起：

```
//: arrays/MultidimensionalObjectArrays.java
import java.util.*;

public class MultidimensionalObjectArrays {
  public static void main(String[] args) {
    BerylliumSphere[][] spheres = {
      { new BerylliumSphere(), new BerylliumSphere() },
      { new BerylliumSphere(), new BerylliumSphere(),
        new BerylliumSphere(), new BerylliumSphere() },
      { new BerylliumSphere(), new BerylliumSphere(),
        new BerylliumSphere(), new BerylliumSphere(),
        new BerylliumSphere(), new BerylliumSphere(),
        new BerylliumSphere(), new BerylliumSphere() },
    };
    System.out.println(Arrays.deepToString(spheres));
```

```
  }
} /* Output:
[[Sphere 0, Sphere 1], [Sphere 2, Sphere 3, Sphere 4,
Sphere 5], [Sphere 6, Sphere 7, Sphere 8, Sphere 9, Sphere
10, Sphere 11, Sphere 12, Sphere 13]]
*///:~
```

756

你可以看到**spheres**是另外一个粗糙数组，其每一个对象列表的长度都是不同的。

自动包装机制对数组初始化器也起作用：

```
//: arrays/AutoboxingArrays.java
import java.util.*;

public class AutoboxingArrays {
  public static void main(String[] args) {
    Integer[][] a = { // Autoboxing:
      { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 },
      { 21, 22, 23, 24, 25, 26, 27, 28, 29, 30 },
      { 51, 52, 53, 54, 55, 56, 57, 58, 59, 60 },
      { 71, 72, 73, 74, 75, 76, 77, 78, 79, 80 },
    };
    System.out.println(Arrays.deepToString(a));
  }
} /* Output:
[[1, 2, 3, 4, 5, 6, 7, 8, 9, 10], [21, 22, 23, 24, 25, 26,
27, 28, 29, 30], [51, 52, 53, 54, 55, 56, 57, 58, 59, 60],
[71, 72, 73, 74, 75, 76, 77, 78, 79, 80]]
*///:~
```

下面的示例展示了可以如何逐个地、部分地构建一个非基本类型的对象数组：

```
//: arrays/AssemblingMultidimensionalArrays.java
// Creating multidimensional arrays.
import java.util.*;

public class AssemblingMultidimensionalArrays {
  public static void main(String[] args) {
    Integer[][] a;
    a = new Integer[3][];
    for(int i = 0; i < a.length; i++) {
      a[i] = new Integer[3];
      for(int j = 0; j < a[i].length; j++)
        a[i][j] = i * j; // Autoboxing
    }
    System.out.println(Arrays.deepToString(a));
  }
} /* Output:
[[0, 0, 0], [0, 1, 2], [0, 2, 4]]
*///:~
```

757

i*j只是为了使置于**Integer**中的值变得有些意思。

Arrays.deepToString()方法对基本类型数组和对象数组都起作用：

```
//: arrays/MultiDimWrapperArray.java
// Multidimensional arrays of "wrapper" objects.
import java.util.*;

public class MultiDimWrapperArray {
  public static void main(String[] args) {
    Integer[][] a1 = { // Autoboxing
      { 1, 2, 3, },
      { 4, 5, 6, },
    };
    Double[][][] a2 = { // Autoboxing
      { { 1.1, 2.2 }, { 3.3, 4.4 } },
      { { 5.5, 6.6 }, { 7.7, 8.8 } },
```

```
      { { 9.9, 1.2 }, { 2.3, 3.4 } },
    };
    String[][] a3 = {
      { "The", "Quick", "Sly", "Fox" },
      { "Jumped", "Over" },
      { "The", "Lazy", "Brown", "Dog", "and", "friend" },
    };
    System.out.println("a1: " + Arrays.deepToString(a1));
    System.out.println("a2: " + Arrays.deepToString(a2));
    System.out.println("a3: " + Arrays.deepToString(a3));
  }
} /* Output:
a1: [[1, 2, 3], [4, 5, 6]]
a2: [[[1.1, 2.2], [3.3, 4.4]], [[5.5, 6.6], [7.7, 8.8]],
[[9.9, 1.2], [2.3, 3.4]]]
a3: [[The, Quick, Sly, Fox], [Jumped, Over], [The, Lazy,
Brown, Dog, and, friend]]
*///:~
```

在**Integer**和**Double**数组中，Java SE5的自动包装机制再次为我们创建了包装器对象。

练习3：(4) 编写一个方法，能够产生二维双精度型数组并加以初始化。数组的容量由方法的形式参数决定，其初值必须落在另外两个形式参数所指定的区间之内。编写第二个方法，打印出第一个方法所产生的数组。在**main()**中通过产生不同容量的数组并打印其内容来测试这两个方法。

758

练习4：(2) 重复前一个练习，但改为三维数组。

练习5：(1) 证明基本类型的多维数组会自动被初始化为**null**。

练习6：(1) 编写一个方法，它接受两个表示二维数组尺寸的**int**参数。这个方法应该这两个根据尺寸参数，创建并填充一个**BerylliumSphere**二维数组。

练习7：(1) 重复前一个练习，但改为三维数组。

16.5 数组与泛型

通常，数组与泛型不能很好地结合。你不能实例化具有参数化类型的数组：

```
Peel<Banana>[] peels = new Peel<Banana>[10]; // Illegal
```

擦除会移除参数类型信息，而数组必须知道它们所持有的确切类型，以强制保证类型安全。

但是，你可以参数化数组本身的类型：

```
//: arrays/ParameterizedArrayType.java

class ClassParameter<T> {
  public T[] f(T[] arg) { return arg; }
}

class MethodParameter {
  public static <T> T[] f(T[] arg) { return arg; }
}

public class ParameterizedArrayType {
  public static void main(String[] args) {
    Integer[] ints = { 1, 2, 3, 4, 5 };
    Double[] doubles = { 1.1, 2.2, 3.3, 4.4, 5.5 };
    Integer[] ints2 =
      new ClassParameter<Integer>().f(ints);
    Double[] doubles2 =
      new ClassParameter<Double>().f(doubles);
    ints2 = MethodParameter.f(ints);
    doubles2 = MethodParameter.f(doubles);
  }
} ///:~
```

759

注意，使用参数化方法而不使用参数化类的方便之处在于：你不必为需要应用的每种不同的类型都使用一个参数去实例化这个类，并且你可以将其定义为静态的。当然，你不能总是选择使用参数化方法而不是参数化类，但是它应该成为首选。

正如上例所证明的那样，不能创建泛型数组这一说法并不十分准确。诚然，编译器确实不让你实例化泛型数组，但是，它允许你创建对这种数组的引用。例如：

```
List<String>[] ls;
```

这条语句可以顺利地通过编译器而不报任何错误。而且，尽管你不能创建实际的持有泛型的数组对象，但是你可以创建非泛型的数组，然后将其转型：

```
//: arrays/ArrayOfGenerics.java
// It is possible to create arrays of generics.
import java.util.*;

public class ArrayOfGenerics {
  @SuppressWarnings("unchecked")
  public static void main(String[] args) {
    List<String>[] ls;
    List[] la = new List[10];
    ls = (List<String>[])la; // "Unchecked" warning
    ls[0] = new ArrayList<String>();
    // Compile-time checking produces an error:
    //! ls[1] = new ArrayList<Integer>();

    // The problem: List<String> is a subtype of Object
    Object[] objects = ls; // So assignment is OK
    // Compiles and runs without complaint:
    objects[1] = new ArrayList<Integer>();

    // However, if your needs are straightforward it is
    // possible to create an array of generics, albeit
    // with an "unchecked" warning:
    List<BerylliumSphere>[] spheres =
      (List<BerylliumSphere>[])new List[10];
    for(int i = 0; i < spheres.length; i++)
      spheres[i] = new ArrayList<BerylliumSphere>();
  }
} ///:~
```

760

一旦拥有了对**List<String>[]**的引用，你就会看到你将得到某些编译器检查。问题是数组是协变类型的，因此**List<String>[]**也是一个**Object[]**，并且你可以利用这一点，将一个**ArrayList<Integer>**赋值到你的数组中，而不会有任何编译期或运行时错误。

如果你知道将来不会向上转型，并且需求也相对比较简单，那么你仍旧可以创建泛型数组，它可以提供基本的编译期类型检查。但是，事实上，泛型容器总是比泛型数据更好的选择。

一般而言，你会发现泛型在类或方法的边界处很有效，而在类或方法的内部，擦除通常会使泛型变得不适用。例如，你不能创建泛型数组：

```
//: arrays/ArrayOfGenericType.java
// Arrays of generic types won't compile.

public class ArrayOfGenericType<T> {
  T[] array; // OK
  @SuppressWarnings("unchecked")
  public ArrayOfGenericType(int size) {
    //! array = new T[size]; // Illegal
    array = (T[])new Object[size]; // "unchecked" Warning
  }
  // Illegal:
  //! public <U> U[] makeArray() { return new U[10]; }
} ///:~
```

擦除再次成为了障碍——本例试图创建的类型已被擦除，因而是类型未知的数组。注意，你可以创建**Object**数组，然后将其转型，但是如果没有**@SuppressWarnings**注解，你将在编译期得到一个“不受检查”的警告消息，因为这个数组没有真正持有或动态检查类型**T**。也就是说，如果我创建一个**String[]**，Java在编译期和运行时都会强制要求我只能将**String**对象置于该数组中。但是，如果创建的是**Object[]**，那么我就可以将除基本类型之外的任何对象置于该数组中。

761

练习8：(1) 证明前一段话中的断言。

练习9：(3) 创建**Peel<Banana>**所必需的类，并展示编译器不会接受它。使用**ArrayList**来改正此问题。

练习10：(2) 修改**ArrayOfGenerics.java**，在其中使用容器而不是数组。展示你可以根除编译期警告信息。

16.6 创建测试数据

通常，在试验数组和程序时，能够很方便地生成填充了测试数据的数组，将会很有帮助。本节介绍的工具就可以用数值或对象来填充数组。

16.6.1 Arrays.fill()

Java标准类库**Arrays**有一个作用十分有限的**fill()**方法：只能用同一个值填充各个位置，而针对对象而言，就是复制同一个引用进行填充。下面是一个示例：

```
//: arrays/FillingArrays.java
// Using Arrays.fill()
import java.util.*;
import static net.mindview.util.Print.*;

public class FillingArrays {
  public static void main(String[] args) {
    int size = 6;
    boolean[] a1 = new boolean[size];
    byte[] a2 = new byte[size];
    char[] a3 = new char[size];
    short[] a4 = new short[size];
    int[] a5 = new int[size];
    long[] a6 = new long[size];
    float[] a7 = new float[size];
    double[] a8 = new double[size];
    String[] a9 = new String[size];
    Arrays.fill(a1, true);
    print("a1 = " + Arrays.toString(a1));
    Arrays.fill(a2, (byte)11);
    print("a2 = " + Arrays.toString(a2));
    Arrays.fill(a3, 'x');
    print("a3 = " + Arrays.toString(a3));
    Arrays.fill(a4, (short)17);
    print("a4 = " + Arrays.toString(a4));
    Arrays.fill(a5, 19);
    print("a5 = " + Arrays.toString(a5));
    Arrays.fill(a6, 23);
    print("a6 = " + Arrays.toString(a6));
    Arrays.fill(a7, 29);
    print("a7 = " + Arrays.toString(a7));
    Arrays.fill(a8, 47);
    print("a8 = " + Arrays.toString(a8));
    Arrays.fill(a9, "Hello");
    print("a9 = " + Arrays.toString(a9));
    // Manipulating ranges:
    Arrays.fill(a9, 3, 5, "World");
    print("a9 = " + Arrays.toString(a9));
```

762

```
  }
} /* Output:
a1 = [true, true, true, true, true, true]
a2 = [11, 11, 11, 11, 11, 11]
a3 = [x, x, x, x, x, x]
a4 = [17, 17, 17, 17, 17, 17]
a5 = [19, 19, 19, 19, 19, 19]
a6 = [23, 23, 23, 23, 23, 23]
a7 = [29.0, 29.0, 29.0, 29.0, 29.0, 29.0]
a8 = [47.0, 47.0, 47.0, 47.0, 47.0, 47.0]
a9 = [Hello, Hello, Hello, Hello, Hello, Hello]
a9 = [Hello, Hello, Hello, World, World, Hello]
*///:~
```

使用**Arrays.fill()**可以填充整个数组，或者像最后两条语句所示，只填充数组的某个区域。但是由于只能用单一的数值来调用**Arrays.fill()**，因此所产生的结果并非特别有用。

16.6.2 数据生成器

为了以灵活的方式创建更有意义的数组，我们将使用在第15章中引入的**Generator**概念。如果某个工具使用了**Generator**，那么你就可以通过选择**Generator**的类型来创建任何类型的数据（这是策略设计模式的一个实例——每个不同的**Generator**都表示一个不同的策略[⊖]）。 763

本节将提供一些**Generator**，并且，就像之前看到的，你还可以很容易地定义自己的**Generator**。

首先给出的是可以用于所有基本类型的包装器类型，以及**String**类型的最基本的计数生成器集合。这些生成器类都嵌套在**CountingGenerator**类中，从而使得它们能够使用与所要生成的对象类型相同的名字。例如，创建**Integer**对象的生成器可以通过表达式**new CountingGenerator.Integer()**来创建：

```
//: net/mindview/util/CountingGenerator.java
// Simple generator implementations.
package net.mindview.util;

public class CountingGenerator {
  public static class
  Boolean implements Generator<java.lang.Boolean> {
    private boolean value = false;
    public java.lang.Boolean next() {
      value = !value; // Just flips back and forth
      return value;
    }
  }
  public static class
  Byte implements Generator<java.lang.Byte> {
    private byte value = 0;
    public java.lang.Byte next() { return value++; }
  }
  static char[] chars = ("abcdefghijklmnopqrstuvwxyz" +
    "ABCDEFGHIJKLMNOPQRSTUVWXYZ").toCharArray();
  public static class
  Character implements Generator<java.lang.Character> {
    int index = -1;
    public java.lang.Character next() {
      index = (index + 1) % chars.length;
      return chars[index];
    }
  }
  public static class
```

764

⊖ 尽管在这里事情变得有点含糊，可能在此会产生争议，认为**Generator**应用了命令模式，但是我认为，实际的任务是填充数组，而**Generator**只实现了该任务的一部分，因此它更像是策略而不是命令。

```
String implements Generator<java.lang.String> {
  private int length = 7;
  Generator<java.lang.Character> cg = new Character();
  public String() {}
  public String(int length) { this.length = length; }
  public java.lang.String next() {
    char[] buf = new char[length];
    for(int i = 0; i < length; i++)
      buf[i] = cg.next();
    return new java.lang.String(buf);
  }
}
public static class
Short implements Generator<java.lang.Short> {
  private short value = 0;
  public java.lang.Short next() { return value++; }
}
public static class
Integer implements Generator<java.lang.Integer> {
  private int value = 0;
  public java.lang.Integer next() { return value++; }
}
public static class
Long implements Generator<java.lang.Long> {
  private long value = 0;
  public java.lang.Long next() { return value++; }
}
public static class
Float implements Generator<java.lang.Float> {
  private float value = 0;
  public java.lang.Float next() {
    float result = value;
    value += 1.0;
    return result;
  }
}
public static class
Double implements Generator<java.lang.Double> {
  private double value = 0.0;
  public java.lang.Double next() {
    double result = value;
    value += 1.0;
    return result;
  }
}
} ///:~
```

765

上面的每个类都实现了某种意义的“计数”。在**CountingGenerator.Character**中，计数只是不断地重复大写和小写字母；**CountingGenerator.String**类使用**CountingGenerator.Character**来填充一个字符数组，该数组将被转换为**String**，数组的尺寸取决于构造器参数。请注意，**CountingGenerator.String**使用的是基本的**Generator<java.lang.Character>**，而不是具体的对**CountingGenerator.Character**的引用。稍后，我们可以替换这个生成器，以生成**RandomGenerator.java**中的**RandomGenerator.String**。

下面是一个测试工具，针对嵌套的**Generator**这一惯用法，因为使用了反射所以这个工具可以遵循下面的形式来测试**Generator**的任何集合。

```
//: arrays/GeneratorsTest.java
import net.mindview.util.*;

public class GeneratorsTest {
  public static int size = 10;
  public static void test(Class<?> surroundingClass) {
    for(Class<?> type : surroundingClass.getClasses()) {
```

```
      System.out.print(type.getSimpleName() + ": ");
      try {
        Generator<?> g = (Generator<?>)type.newInstance();
        for(int i = 0; i < size; i++)
          System.out.printf(g.next() + " ");
        System.out.println();
      } catch(Exception e) {
        throw new RuntimeException(e);
      }
    }
  }
  public static void main(String[] args) {
    test(CountingGenerator.class);
  }
} /* Output:
Double: 0.0 1.0 2.0 3.0 4.0 5.0 6.0 7.0 8.0 9.0
Float: 0.0 1.0 2.0 3.0 4.0 5.0 6.0 7.0 8.0 9.0
Long: 0 1 2 3 4 5 6 7 8 9
Integer: 0 1 2 3 4 5 6 7 8 9
Short: 0 1 2 3 4 5 6 7 8 9
String: abcdefg hijklmn opqrstu vwxyzAB CDEFGHI JKLMNOP
QRSTUVW XYZabcd efghijk lmnopqr
Character: a b c d e f g h i j
Byte: 0 1 2 3 4 5 6 7 8 9
Boolean: true false true false true false true false true
false
*///:~
```

766

这里假设待测类包含一组嵌套的**Generator**对象，其中每个都有一个默认构造器（无参构造器）。反射方法**getClasses()**可以生成所有的嵌套类，而**test()**方法可以为这些生成器中的每一个都创建一个实例，然后打印通过调用10次**next()**方法而产生的结果。

下面是一组使用随机数生成器的**Generator**。因为**Random**构造器使用常量进行初始化，所以，每次用这些**Generator**中的一个来运行程序时，所产生的输出都是可重复的：

```
//: net/mindview/util/RandomGenerator.java
// Generators that produce random values.
package net.mindview.util;
import java.util.*;

public class RandomGenerator {
  private static Random r = new Random(47);
  public static class
  Boolean implements Generator<java.lang.Boolean> {
    public java.lang.Boolean next() {
      return r.nextBoolean();
    }
  }
  public static class
  Byte implements Generator<java.lang.Byte> {
    public java.lang.Byte next() {
      return (byte)r.nextInt();
    }
  }
  public static class
  Character implements Generator<java.lang.Character> {
    public java.lang.Character next() {
      return CountingGenerator.chars[
        r.nextInt(CountingGenerator.chars.length)];
    }
  }
  public static class
  String extends CountingGenerator.String {
    // Plug in the random Character generator:
    { cg = new Character(); } // Instance initializer
    public String() {}
```

767

```
    public String(int length) { super(length); }
  }
  public static class
  Short implements Generator<java.lang.Short> {
    public java.lang.Short next() {
      return (short)r.nextInt();
    }
  }
  public static class
  Integer implements Generator<java.lang.Integer> {
    private int mod = 10000;
    public Integer() {}
    public Integer(int modulo) { mod = modulo; }
    public java.lang.Integer next() {
      return r.nextInt(mod);
    }
  }
  public static class
  Long implements Generator<java.lang.Long> {
    private int mod = 10000;
    public Long() {}
    public Long(int modulo) { mod = modulo; }
    public java.lang.Long next() {
      return new java.lang.Long(r.nextInt(mod));
    }
  }
  public static class
  Float implements Generator<java.lang.Float> {
    public java.lang.Float next() {
      // Trim all but the first two decimal places:
      int trimmed = Math.round(r.nextFloat() * 100);
      return ((float)trimmed) / 100;
    }
  }
  public static class
  Double implements Generator<java.lang.Double> {
    public java.lang.Double next() {
      long trimmed = Math.round(r.nextDouble() * 100);
      return ((double)trimmed) / 100;
    }
  }
} ///:~
```

768

你可以看到，**RandomGenerator.String**继承自**CountingGenerator.String**，并且只是插入了新的**Character**生成器。

为了不生成过大的数字，**RandomGenerator.Integer**默认使用的模数为10 000，但是重载的构造器允许你选择更小的值。同样的方式也应用到了**RandomGenerator.Long**上。对于**Float**和**Double**生成器，小数点之后的数字被截掉了。

我们复用**GeneratorTest**来测试**RandomGenerator**：

```
//: arrays/RandomGeneratorsTest.java
import net.mindview.util.*;

public class RandomGeneratorsTest {
  public static void main(String[] args) {
    GeneratorsTest.test(RandomGenerator.class);
  }
} /* Output:
Double: 0.73 0.53 0.16 0.19 0.52 0.27 0.26 0.05 0.8 0.76
Float: 0.53 0.16 0.53 0.4 0.49 0.25 0.8 0.11 0.02 0.8
Long: 7674 8804 8950 7826 4322 896 8033 2984 2344 5810
Integer: 8303 3141 7138 6012 9966 8689 7185 6992 5746 3976
Short: 3358 20592 284 26791 12834 -8092 13656 29324 -1423
5327
```

```
String: bkInaMe sbtWHkj UrUkZPg wsqPzDy CyRFJQA HxxHvHq
XumcXZJ oogoYWM NvqeuTp nXsgqia
Character: x x E A J J m z M s
Byte: -60 -17 55 -14 -5 115 39 -37 79 115
Boolean: false true false false true true true true true
true
*///:~
```

你可以通过修改**public**的**GeneratorTest.size**的值，来改变所产生的数值数量。 769

16.6.3 从Generator中创建数组

为了接收**Generator**并产生数组，我们需要两个转换工具。第一个工具使用任意的**Generator**来产生**Object**子类型的数组。为了处理基本类型，第二个工具接收任意基本类型的包装器类型数组，并产生相应的基本类型数组。

第一个工具有两种选项，并由重载的静态方法**array()**来表示。该方法的第一个版本接收一个已有的数组，并使用某个**Generator**来填充它，而第二个版本接收一个**Class**对象、一个**Generator**和所需的元素数量，然后创建一个新数组，并使用所接收的**Generator**来填充它。注意，这个工具只能产生**Object**子类型的数组，而不能产生基本类型数组：

```
//: net/mindview/util/Generated.java
package net.mindview.util;
import java.util.*;

public class Generated {
  // Fill an existing array:
  public static <T> T[] array(T[] a, Generator<T> gen) {
    return new CollectionData<T>(gen, a.length).toArray(a);
  }
  // Create a new array:
  @SuppressWarnings("unchecked")
  public static <T> T[] array(Class<T> type,
      Generator<T> gen, int size) {
    T[] a =
      (T[])java.lang.reflect.Array.newInstance(type, size);
    return new CollectionData<T>(gen, size).toArray(a);
  }
} ///:~
```

CollectionData类将在第17章中定义，它将创建一个**Collection**对象，该对象中所填充的元素是由生成器**gen**产生的，而元素的数量则由构造器的第二个参数确定。所有的**Collection**子类型都拥有**toArray()**方法，该方法将使用**Collection**中的元素来填充参数数组。 770

第二个方法使用反射来动态创建具有恰当类型和数量的新数组，然后使用与第一个方法相同的技术来填充该数组。

我们可以使用在前一节中定义的**CountingGenerator**类中的某个生成器来测试**Generated**：

```
//: arrays/TestGenerated.java
import java.util.*;
import net.mindview.util.*;

public class TestGenerated {
  public static void main(String[] args) {
    Integer[] a = { 9, 8, 7, 6 };
    System.out.println(Arrays.toString(a));
    a = Generated.array(a,new CountingGenerator.Integer());
    System.out.println(Arrays.toString(a));
    Integer[] b = Generated.array(Integer.class,
        new CountingGenerator.Integer(), 15);
    System.out.println(Arrays.toString(b));
  }
} /* Output:
```

```
[9, 8, 7, 6]
[0, 1, 2, 3]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
*///:~
```

即使数组a被初始化过，其中的那些值也在将其传递给**Generated.array()**之后被覆写了，因为这个方法会替换这些值（但是会保证原数组的正确性）。**b**的初始化展示了如何从无到有地创建填充了元素的数组。

泛型不能用于基本类型，而我们确实想用生成器来填充基本类型数组。为了解决这个问题，我们创建了一个转换器，它可以接收任意的包装器对象数组，并将其转换为相应的基本类型数组。如果没有这个工具，我们就必须为所有的基本类型创建特殊的生成器。

```
//: net/mindview/util/ConvertTo.java
package net.mindview.util;

public class ConvertTo {
  public static boolean[] primitive(Boolean[] in) {
    boolean[] result = new boolean[in.length];
    for(int i = 0; i < in.length; i++)
      result[i] = in[i]; // Autounboxing
    return result;
  }
  public static char[] primitive(Character[] in) {
    char[] result = new char[in.length];
    for(int i = 0; i < in.length; i++)
      result[i] = in[i];
    return result;
  }
  public static byte[] primitive(Byte[] in) {
    byte[] result = new byte[in.length];
    for(int i = 0; i < in.length; i++)
      result[i] = in[i];
    return result;
  }
  public static short[] primitive(Short[] in) {
    short[] result = new short[in.length];
    for(int i = 0; i < in.length; i++)
      result[i] = in[i];
    return result;
  }
  public static int[] primitive(Integer[] in) {
    int[] result = new int[in.length];
    for(int i = 0; i < in.length; i++)
      result[i] = in[i];
    return result;
  }
  public static long[] primitive(Long[] in) {
    long[] result = new long[in.length];
    for(int i = 0; i < in.length; i++)
      result[i] = in[i];
    return result;
  }
  public static float[] primitive(Float[] in) {
    float[] result = new float[in.length];
    for(int i = 0; i < in.length; i++)
      result[i] = in[i];
    return result;
  }
  public static double[] primitive(Double[] in) {
    double[] result = new double[in.length];
    for(int i = 0; i < in.length; i++)
      result[i] = in[i];
    return result;
  }
} ///:~
```

771
772

primitive()方法的每个版本都可以创建适当的具有恰当长度的基本类型数组，然后向其中复制包装器类型数组**in**中的元素。注意，在下面的表达式中会进行自动拆包：

```
result[i] = in[i];
```

下面的示例展示了如何将**ConvertTo**应用于两个版本的**Generated.array()**上：

```
//: arrays/PrimitiveConversionDemonstration.java
import java.util.*;
import net.mindview.util.*;

public class PrimitiveConversionDemonstration {
  public static void main(String[] args) {
    Integer[] a = Generated.array(Integer.class,
        new CountingGenerator.Integer(), 15);
    int[] b = ConvertTo.primitive(a);
    System.out.println(Arrays.toString(b));
    boolean[] c = ConvertTo.primitive(
      Generated.array(Boolean.class,
        new CountingGenerator.Boolean(), 7));
    System.out.println(Arrays.toString(c));
  }
} /* Output:
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
[true, false, true, false, true, false, true]
*///:~
```

最后，下面的程序将使用**RandomGenerator**中的类来测试这些数组生成工具：

```
//: arrays/TestArrayGeneration.java
// Test the tools that use generators to fill arrays.
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

public class TestArrayGeneration {
  public static void main(String[] args) {
    int size = 6;
    boolean[] a1 = ConvertTo.primitive(Generated.array(
      Boolean.class, new RandomGenerator.Boolean(), size));
    print("a1 = " + Arrays.toString(a1));
    byte[] a2 = ConvertTo.primitive(Generated.array(
      Byte.class, new RandomGenerator.Byte(), size));
    print("a2 = " + Arrays.toString(a2));
    char[] a3 = ConvertTo.primitive(Generated.array(
      Character.class,
      new RandomGenerator.Character(), size));
    print("a3 = " + Arrays.toString(a3));
    short[] a4 = ConvertTo.primitive(Generated.array(
      Short.class, new RandomGenerator.Short(), size));
    print("a4 = " + Arrays.toString(a4));
    int[] a5 = ConvertTo.primitive(Generated.array(
      Integer.class, new RandomGenerator.Integer(), size));
    print("a5 = " + Arrays.toString(a5));
    long[] a6 = ConvertTo.primitive(Generated.array(
      Long.class, new RandomGenerator.Long(), size));
    print("a6 = " + Arrays.toString(a6));
    float[] a7 = ConvertTo.primitive(Generated.array(
      Float.class, new RandomGenerator.Float(), size));
    print("a7 = " + Arrays.toString(a7));
    double[] a8 = ConvertTo.primitive(Generated.array(
      Double.class, new RandomGenerator.Double(), size));
    print("a8 = " + Arrays.toString(a8));
  }
} /* Output:
a1 = [true, false, true, false, false, true]
a2 = [104, -79, -76, 126, 33, -64]
```

773

```
a3 = [Z, n, T, c, Q, r]
a4 = [-13408, 22612, 15401, 15161, -28466, -12603]
a5 = [7704, 7383, 7706, 575, 8410, 6342]
a6 = [7674, 8804, 8950, 7826, 4322, 896]
a7 = [0.01, 0.2, 0.4, 0.79, 0.27, 0.45]
a8 = [0.16, 0.87, 0.7, 0.66, 0.87, 0.59]
*///:~
```

这些测试还可以确保**ConvertTo.primitive()**方法的每个版本都可以正确地工作。

练习11：(2) 展示自动包装机制不能应用于数组。

练习12：(1) 用**CountingGenerator**创建一个初始化过的**double**数组并打印结果。

练习13：(2) 用**CountingGenerator.Character**填充一个**String**。

练习14：(6) 对每个基本类型都创建一个数组，然后用**CountingGenerator**来填充每个数组并打印所有的数组。

774

练习15：(2) 修改**ContainerComparison.java**，创建一个用于**BerylliumSphere**的**Generator**，并修改**main()**方法，再将这个**Generator**作用于**Generated.array()**。

练习16：(3) 从**CountingGenerator.java**开始，创建一个**SkipGenerator**类，它可以根据构造器参数，通过递增产生新值。修改**TestArrayGeneration.java**，以展示新类可以正确地工作。

练习17：(5) 创建并测试用于**BigDecimal**的**Generator**，并确保它可以用于**Generated**中的方法。

16.7 Arrays实用功能

在**java.util**类库中可以找到**Arrays**类，它有一套用于数组的**static**实用方法，其中有六个基本方法：**equals()**用于比较两个数组是否相等（**deepEquals()**用于多维数组）；**fill()**在本章前面部分已经论述过了；**sort()**用于对数组排序；**binarySearch()**用于在已经排序的数组中查找元素；**toString()**产生数组的**String**表示；**hashCode()**产生数组的散列码（你将在第17章中学习它）。所有这些方法对各种基本类型和**Object**类而重载过。此外，**Arrays.asList()**接受任意的序列或数组作为其参数，并将其转变为**List**容器——这个方法在第11章中已经介绍过了。

在讨论**Arrays**的方法之前，我们先看看另一个不属于**Arrays**但很有用的方法。

16.7.1 复制数组

Java标准类库提供有**static**方法**System.arraycopy()**，用它复制数组比用**for**循环复制要快很多。**System.arraycopy()**针对所有类型做了重载。下面的例子就是用来处理**int**数组的：

```
//: arrays/CopyingArrays.java
// Using System.arraycopy()
import java.util.*;
import static net.mindview.util.Print.*;

public class CopyingArrays {
  public static void main(String[] args) {
    int[] i = new int[7];
    int[] j = new int[10];
    Arrays.fill(i, 47);
    Arrays.fill(j, 99);
    print("i = " + Arrays.toString(i));
    print("j = " + Arrays.toString(j));
    System.arraycopy(i, 0, j, 0, i.length);
    print("j = " + Arrays.toString(j));
    int[] k = new int[5];
    Arrays.fill(k, 103);
```

775

```
    System.arraycopy(i, 0, k, 0, k.length);
    print("k = " + Arrays.toString(k));
    Arrays.fill(k, 103);
    System.arraycopy(k, 0, i, 0, k.length);
    print("i = " + Arrays.toString(i));
    // Objects:
    Integer[] u = new Integer[10];
    Integer[] v = new Integer[5];
    Arrays.fill(u, new Integer(47));
    Arrays.fill(v, new Integer(99));
    print("u = " + Arrays.toString(u));
    print("v = " + Arrays.toString(v));
    System.arraycopy(v, 0, u, u.length/2, v.length);
    print("u = " + Arrays.toString(u));
  }
} /* Output:
i = [47, 47, 47, 47, 47, 47, 47]
j = [99, 99, 99, 99, 99, 99, 99, 99, 99, 99]
j = [47, 47, 47, 47, 47, 47, 47, 99, 99, 99]
k = [47, 47, 47, 47, 47]
i = [103, 103, 103, 103, 103, 47, 47]
u = [47, 47, 47, 47, 47, 47, 47, 47, 47, 47]
v = [99, 99, 99, 99, 99]
u = [47, 47, 47, 47, 47, 99, 99, 99, 99, 99]
*///:~
```

arraycopy()需要的参数有：源数组，表示从源数组中的什么位置开始复制的偏移量，表示从目标数组的什么位置开始复制的偏移量，以及需要复制的元素个数。当然，对数组的任何越界操作都会导致异常。

这个例子说明基本类型数组与对象数组都可以复制。然而，如果复制对象数组，那么只是复制了对象的引用——而不是对象本身的拷贝。这被称作浅复制（shallow copy）（参见本书的在线补充材料以了解更多的内容）。 776

System.arraycopy()不会执行自动包装和自动拆包，两个数组必须具有相同的确切类型。

练习18：(3) 创建并填充一个**BerylliumSphere**数组，将这个数组复制到一个新数组中，并展示这是一种浅复制。

16.7.2 数组的比较

Arrays类提供了重载后的**equals()**方法，用来比较整个数组。同样，此方法针对所有基本类型与**Object**都做了重载。数组相等的条件是元素个数必须相等，并且对应位置的元素也相等，这可以通过对每一个元素使用**equals()**作比较来判断。（对于基本类型，需要使用基本类型的包装器类的**equals()**方法，例如，对于**int**类型使用**Integer.equals()**作比较）见下例：

```
//: arrays/ComparingArrays.java
// Using Arrays.equals()
import java.util.*;
import static net.mindview.util.Print.*;

public class ComparingArrays {
  public static void main(String[] args) {
    int[] a1 = new int[10];
    int[] a2 = new int[10];
    Arrays.fill(a1, 47);
    Arrays.fill(a2, 47);
    print(Arrays.equals(a1, a2));
    a2[3] = 11;
    print(Arrays.equals(a1, a2));
    String[] s1 = new String[4];
    Arrays.fill(s1, "Hi");
```

```
    String[] s2 = { new String("Hi"), new String("Hi"),
      new String("Hi"), new String("Hi") };
    print(Arrays.equals(s1, s2));
  }
} /* Output:
true
false
true
*///:~
```

777

最初，**a1**与**a2**完全相等，所以输出为true；然后改变其中一个元素，使得结果为false。在最后一个例子中，**s1**的所有元素都指向同一个对象，而数组**s2**包含五个相互独立的对象。然而，数组相等是基于内容的（通过**Object.equals()**比较），所以结果为true。

练习19：(2) 创建一个类，它有一个用构造器中的参数初始化的**int**域。创建由这个类的对象构成的两个数组，每个数组都使用了相同的初始化值，然后展示它们不相等的**Arrays.equals()**声明。在你的类中添加一个**equals()**方法来解决此问题。

练习20：(4) 演示用于多维数组的**deepEquals()**方法。

16.7.3 数组元素的比较

排序必须根据对象的实际类型执行比较操作。一种自然的解决方案是为每种不同的类型各编写一个不同的排序方法，但是这样的代码难以被新的类型所复用。

程序设计的基本目标是“将保持不变的事物与会发生改变的事物相分离”，而这里，不变的是通用的排序算法，变化的是各种对象相互比较的方式。因此，不是将进行比较的代码编写成不同的子程序，而是使用策略设计模式[⊖]。通过使用策略，可以将“会发生变化的代码”封装在单独的类中（策略对象），你可以将策略对象传递给总是相同的代码，这些代码将使用策略来完成其算法。通过这种方式，你能够用不同的对象来表示不同的比较方式，然后将它们传递给相同的排序代码。

Java 有两种方式来提供比较功能。第一种是实现**java.lang.Comparable**接口，使你的类具有“天生”的比较能力。此接口很简单，只有**compareTo()**一个方法。此方法接收另一个**Object**为参数，如果当前对象小于参数则返回负值，如果相等则返回零，如果当前对象大于参数则返回正值。

778

下面的类实现了**Comparable**接口，并且使用Java标准类库的方法**Arrays.sort()**演示了比较的效果：

```
//: arrays/CompType.java
// Implementing Comparable in a class.
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

public class CompType implements Comparable<CompType> {
  int i;
  int j;
  private static int count = 1;
  public CompType(int n1, int n2) {
    i = n1;
    j = n2;
  }
  public String toString() {
    String result = "[i = " + i + ", j = " + j + "]";
    if(count++ % 3 == 0)
```

⊖ 参考Erich Gamma的《Design Pattern》和www.MindView上的《Thinking in Pattern（With Java）》。其中《Design Pattern》的中文版、英文影印版及双语版均已由机械工业出版社出版——编辑注。

```
      result += "\n";
    return result;
  }
  public int compareTo(CompType rv) {
    return (i < rv.i ? -1 : (i == rv.i ? 0 : 1));
  }
  private static Random r = new Random(47);
  public static Generator<CompType> generator() {
    return new Generator<CompType>() {
      public CompType next() {
        return new CompType(r.nextInt(100),r.nextInt(100));
      }
    };
  }
  public static void main(String[] args) {
    CompType[] a =
      Generated.array(new CompType[12], generator());
    print("before sorting:");
    print(Arrays.toString(a));
    Arrays.sort(a);
    print("after sorting:");
    print(Arrays.toString(a));
  }
} /* Output:
before sorting:
[[i = 58, j = 55], [i = 93, j = 61], [i = 61, j = 29]
, [i = 68, j = 0], [i = 22, j = 7], [i = 88, j = 28]
, [i = 51, j = 89], [i = 9, j = 78], [i = 98, j = 61]
, [i = 20, j = 58], [i = 16, j = 40], [i = 11, j = 22]
]
after sorting:
[[i = 9, j = 78], [i = 11, j = 22], [i = 16, j = 40]
, [i = 20, j = 58], [i = 22, j = 7], [i = 51, j = 89]
, [i = 58, j = 55], [i = 61, j = 29], [i = 68, j = 0]
, [i = 88, j = 28], [i = 93, j = 61], [i = 98, j = 61]
]
*///:~
```

779

在定义作比较的方法时，由你来负责决定将你的一个对象与另一个对象作比较的含义。这里在比较中只用到了**i**值，而忽略了**j**值。

generator()方法生成一个对象，此对象通过创建一个匿名内部类（见第8章）来实现**Generator**接口。该例中构建**CompType**对象，并使用随机数加以初始化。在**main()**中，使用生成器填充**CompType**的数组，然后对其排序。如果没有实现**Comparable**接口，调用**sort()**的时候会抛出**ClassCastException**这个运行时异常。因为**sort()**需要把参数的类型转变为**Comparable**。

假设有人给你一个并没有实现**Comparable**的类，或者给你的类实现了**Comparable**，但是你不喜欢它的实现方式，你需要另外一种不同的比较方法。要解决这个问题，可以创建一个实现了**Comparator**接口（在第11章中简要介绍过）的单独的类。这是策略设计模式的一个应用实例。这个类有**compare()** 和**equals()** 两个方法。然而，不一定要实现**equals()**方法，除非有特殊的性能需要，因为无论何时创建一个类，都是间接继承自**Object**，而**Object**带有**equals()**方法。所以只需用默认的**Object**的**equals()**方法就可以满足接口的要求了。

780

Collections类（在下一章会详细介绍）包含一个**reverseOrder()**方法，该方法可以产生一个**Comparator**，它可以反转自然的排序顺序。这很容易应用于**CompType**:

```
//: arrays/Reverse.java
// The Collections.reverseOrder() Comparator
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

public class Reverse {
```

```
  public static void main(String[] args) {
    CompType[] a = Generated.array(
      new CompType[12], CompType.generator());
    print("before sorting:");
    print(Arrays.toString(a));
    Arrays.sort(a, Collections.reverseOrder());
    print("after sorting:");
    print(Arrays.toString(a));
  }
} /* Output:
before sorting:
[[i = 58, j = 55], [i = 93, j = 61], [i = 61, j = 29]
, [i = 68, j = 0], [i = 22, j = 7], [i = 88, j = 28]
, [i = 51, j = 89], [i = 9, j = 78], [i = 98, j = 61]
, [i = 20, j = 58], [i = 16, j = 40], [i = 11, j = 22]
]
after sorting:
[[i = 98, j = 61], [i = 93, j = 61], [i = 88, j = 28]
, [i = 68, j = 0], [i = 61, j = 29], [i = 58, j = 55]
, [i = 51, j = 89], [i = 22, j = 7], [i = 20, j = 58]
, [i = 16, j = 40], [i = 11, j = 22], [i = 9, j = 78]
]
*///:~
```

也可以编写自己的Comparator。在这里的CompType对象是基于j值而不是基于i值的。

```
//: arrays/ComparatorTest.java
// Implementing a Comparator for a class.
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

class CompTypeComparator implements Comparator<CompType> {
  public int compare(CompType o1, CompType o2) {
    return (o1.j < o2.j ? -1 : (o1.j == o2.j ? 0 : 1));
  }
}

public class ComparatorTest {
  public static void main(String[] args) {
    CompType[] a = Generated.array(
      new CompType[12], CompType.generator());
    print("before sorting:");
    print(Arrays.toString(a));
    Arrays.sort(a, new CompTypeComparator());
    print("after sorting:");
    print(Arrays.toString(a));
  }
} /* Output:
before sorting:
[[i = 58, j = 55], [i = 93, j = 61], [i = 61, j = 29]
, [i = 68, j = 0], [i = 22, j = 7], [i = 88, j = 28]
, [i = 51, j = 89], [i = 9, j = 78], [i = 98, j = 61]
, [i = 20, j = 58], [i = 16, j = 40], [i = 11, j = 22]
]
after sorting:
[[i = 68, j = 0], [i = 22, j = 7], [i = 11, j = 22]
, [i = 88, j = 28], [i = 61, j = 29], [i = 16, j = 40]
, [i = 58, j = 55], [i = 20, j = 58], [i = 93, j = 61]
, [i = 98, j = 61], [i = 9, j = 78], [i = 51, j = 89]
]
*///:~
```

781

练习21：(3) 试着对练习18中的对象数组进行排序。

16.7.4 数组排序

使用内置的排序方法，就可以对任意的基本类型数组排序；也可以对任意的对象数组进行

排序，只要该对象实现了**Comparable**接口或具有相关联的**Comparator**[⊖]。下面的例子生成随机的**String**对象，并对其排序：

782

```
//: arrays/StringSorting.java
// Sorting an array of Strings.
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

public class StringSorting {
  public static void main(String[] args) {
    String[] sa = Generated.array(new String[20],
      new RandomGenerator.String(5));
    print("Before sort: " + Arrays.toString(sa));
    Arrays.sort(sa);
    print("After sort: " + Arrays.toString(sa));
    Arrays.sort(sa, Collections.reverseOrder());
    print("Reverse sort: " + Arrays.toString(sa));
    Arrays.sort(sa, String.CASE_INSENSITIVE_ORDER);
    print("Case-insensitive sort: " + Arrays.toString(sa));
  }
} /* Output:
Before sort: [YNzbr, nyGcF, OWZnT, cQrGs, eGZMm, JMRoE,
suEcU, OneOE, dLsmw, HLGEa, hKcxr, EqUCB, bkIna, Mesbt,
WHkjU, rUkZP, gwsqP, zDyCy, RFJQA, HxxHv]
After sort: [EqUCB, HLGEa, HxxHv, JMRoE, Mesbt, OWZnT,
OneOE, RFJQA, WHkjU, YNzbr, bkIna, cQrGs, dLsmw, eGZMm,
gwsqP, hKcxr, nyGcF, rUkZP, suEcU, zDyCy]
Reverse sort: [zDyCy, suEcU, rUkZP, nyGcF, hKcxr, gwsqP,
eGZMm, dLsmw, cQrGs, bkIna, YNzbr, WHkjU, RFJQA, OneOE,
OWZnT, Mesbt, JMRoE, HxxHv, HLGEa, EqUCB]
Case-insensitive sort: [bkIna, cQrGs, dLsmw, eGZMm, EqUCB,
gwsqP, hKcxr, HLGEa, HxxHv, JMRoE, Mesbt, nyGcF, OneOE,
OWZnT, RFJQA, rUkZP, suEcU, WHkjU, YNzbr, zDyCy]
*///:~
```

注意，**String**排序算法依据词典编排顺序排序，所以大写字母开头的词都放在前面输出，然后才是小写字母开头的词。（电话簿通常是这样排序的。）如果想忽略大小写字母将单词都放在一起排序，那么可以像上例中最后一个对**sort()**的调用一样，使用String。CASE_INSENSITIVE_ORDER。

Java标准类库中的排序算法针对正排序的特殊类型进行了优化——针对基本类型设计的“快速排序”（Quicksort），以及针对对象设计的“稳定归并排序”。所以无须担心排序的性能，除非你可以证明排序部分的确是程序效率的瓶颈。 783

16.7.5 在已排序的数组中查找

如果数组已经排好序了，就可以使用**Arrays.binarySearch()**执行快速查找。如果要对未排序的数组使用binarySearch()，那么将产生不可预料的结果。下面的例子使用**RandIntGenerator.Integer**填充数组，然后再使用同样的生成器生成要查找的值：

```
//: arrays/ArraySearching.java
// Using Arrays.binarySearch().
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

public class ArraySearching {
  public static void main(String[] args) {
```

⊖ 令人惊奇的是，Java 1.0或1.1对**String**排序未提供任何支持。

```
    Generator<Integer> gen =
      new RandomGenerator.Integer(1000);
    int[] a = ConvertTo.primitive(
      Generated.array(new Integer[25], gen));
    Arrays.sort(a);
    print("Sorted array: " + Arrays.toString(a));
    while(true) {
      int r = gen.next();
      int location = Arrays.binarySearch(a, r);
      if(location >= 0) {
        print("Location of " + r + " is " + location +
          ", a[" + location + "] = " + a[location]);
        break; // Out of while loop
      }
    }
  }
} /* Output:
Sorted array: [128, 140, 200, 207, 258, 258, 278, 288, 322,
429, 511, 520, 522, 551, 555, 589, 693, 704, 809, 861, 861,
868, 916, 961, 998]
Location of 322 is 8, a[8] = 322
*///:~
```

784 在**while**循环中随机生成一些值作为查找的对象，直到找到一个才停止循环。

如果找到了目标，**Arrays.binarySearch()**产生的返回值等于或大于0。否则，它产生负返回值，表示若要保持数组的排序状态此目标元素所应该插入的位置。这个负值的计算方式是：

-（插入点） - 1

“插入点”是指，第一个大于查找对象的元素在数组中的位置，如果数组中所有的元素都小于要查找的对象，“插入点”就等于**a.size()**。

如果数组包含重复的元素，则无法保证找到的是这些副本中的哪一个。搜索算法确实不是专为包含重复元素的数组而设计的，不过仍然可用。如果需要对没有重复元素的数组排序，可以使用**TreeSet**（保持排序顺序），或者**LinkedHashSet**（保持插入顺序），后面我们将会介绍它们。这些类会自动处理所有的细节。除非它们成为程序性能的瓶颈，否则不需要自己维护数组。

如果使用**Comparator**排序了某个对象数组（基本类型数组无法使用**Comparator**进行排序），在使用**binarySearch()**时必须提供同样的**Comparator**（使用**binarySearch ()**方法的重载版本）。例如，可以修改**StringSorting.java**程序以进行某种查找：

```
//: arrays/AlphabeticSearch.java
// Searching with a Comparator.
import java.util.*;
import net.mindview.util.*;

public class AlphabeticSearch {
  public static void main(String[] args) {
    String[] sa = Generated.array(new String[30],
      new RandomGenerator.String(5));
    Arrays.sort(sa, String.CASE_INSENSITIVE_ORDER);
    System.out.println(Arrays.toString(sa));
    int index = Arrays.binarySearch(sa, sa[10],
      String.CASE_INSENSITIVE_ORDER);
    System.out.println("Index: "+ index + "\n"+ sa[index]);
  }
} /* Output:
[bkIna, cQrGs, cXZJo, dLsmw, eGZMm, EqUCB, gwsqP, hKcxr,
HLGEa, HqXum, HxxHv, JMRoE, JmzMs, Mesbt, MNvqe, nyGcF,
ogoYW, OneOE, OWZnT, RFJQA, rUkZP, sgqia, slJrL, suEcU,
uTpnX, vpfFv, WHkjU, xxEAJ, YNzbr, zDyCy]
Index: 10
HxxHv
*///:~
```

785

这里的**Comparator**必须接受重载过的**binarySearch()**作为其第三个参数。在这个例子中，由于要查找的目标就是从数组中选出来的元素，所以肯定能查找到。

练习22：(2) 通过程序说明在未排序数组上执行**binarySearch()**方法的结果是不可预知的。

练习23：(2) 创建一个**Integer**数组，用随机的**int**数值填充它（使用自动包装机制），再使用**Comparator**将其进行反向排序。

练习24：(3) 通过程序说明练习19中的类是可查找的。

16.8 总结

在本章中，你看到了Java对尺寸固定的低级数组提供了适度的支持。这种数组强调的是性能而不是灵活性，并且与C和C++的数组模型类似。在Java的初始版本中，尺寸固定的低级数组绝对是必需的，不仅是因为Java的设计者选择在Java中要包含基本类型（也是出于性能方面的考虑），而且还因为那个版本中对容器的支持非常少。因此，在Java的早期版本中，选择包含数组总是合理的。

其后的Java版本对容器的支持得到了明显的改进，并且现在的容器在除了性能之外的各个方面都使得数组相形见绌。就像在本书其他多处地方所叙述的那样，对你来说，性能出问题的地方通常是无论如何你都无法想象得到的。

有了额外的自动包装机制和泛型，在容器中持有基本类型就变得易如反掌了，而这也进一步促使你用容器来替换数组。因为泛型可以产生类型安全的容器，因此数组面对这一变化，已经变得毫无优势了。

就像在本章中描述的，而且当你尝试着使用它们时也会看到，泛型对数组是极大的威胁。通常，即使当你可以让泛型与数组以某种方式一起工作时（在下一章你将会看到），在编译期你最终也会得到“不受检查”的警告信息。 786

曾经在多个场合，当我和Java语言的设计者们讨论某些特定的示例时，我直接告诉他们：我应该使用容器而不是数组（在这些示例中，我使用数组是为了演示某些具体的技术，因此我没有选择的余地）。

所有这些话题都表示：当你使用最近的Java版本编程时，应该“优选容器而不是数组”。只有在已证明性能成为问题（并且切换到数组对性能提高有所帮助）时，你才应该将程序重构为使用数组。

这是一个相当清晰的陈述，但是有些语言根本就没有尺寸固定的低级数组，它们只有尺寸可调的容器，这些容器与C/C++/Java风格的数组相比，明显具有更多的功能。例如，Python[⊖]具有一个使用基本数组语法的**List**类型，但是它具有更多功能——你甚至可以继承它。

```
#: arrays/PythonLists.py

aList = [1, 2, 3, 4, 5]
print type(aList) # <type 'list'>
print aList # [1, 2, 3, 4, 5]
print aList[4] # 5   Basic list indexing
aList.append(6) # lists can be resized
aList += [7, 8] # Add a list to a list
print aList # [1, 2, 3, 4, 5, 6, 7, 8]
aSlice = aList[2:4]
print aSlice # [3, 4]

class MyList(list): # Inherit from list
```

⊖ 查看 www.Python.org。

```
    # Define a method, 'this' pointer is explicit:
    def getReversed(self):
        reversed = self[:] # Copy list using slices
        reversed.reverse() # Built-in list method
        return reversed

list2 = MyList(aList) # No 'new' needed for object creation
print type(list2) # <class '__main__.MyList'>
print list2.getReversed() # [8, 7, 6, 5, 4, 3, 2, 1]
#:~
```

787

前一章已经介绍过基本的Python语法。在本例中，直接通过用方括号括起来的且由逗号分割的对象序列，创建了一个列表，所产生的结果就是运行时类型为**List**的一个对象（**print**语句的输出如同一行的注释所示）。打印**List**的结果与使用Java中的**Arrays.toString()**相同。

创建**List**的子序列是通过在索引操作的内部放置“**:**”操作符，从而用“切片”来实现的。**List**类型具有很多内置的操作。

MyList是一个类定义，在括号内的是其基类。在这个类的内部，**def**语句将产生方法，而方法的第一个参数自动地与Java中的**this**等价，只是在Python中它是显式的，并且按惯例其标识符为**self**（这不是关键字）。请注意构造器是如何自动继承的。

尽管Python中的每项事物确实都是对象（包括整型和浮点类型），但是仍旧有其他出口，使得你可以去优化代码中性能关键的部分，这时需要用C或C++编写一些扩展，或者使用被称为Pyrex的特殊工具，它被设计用来让我们更方便地提高代码的执行速度。通过这种方式，你仍旧可以保持对象的纯粹性，同时又不妨碍对性能的改进。

PHP语言[⊖]走得更远，它只有单一的数组类型，即可以充当用**int**来索引的数组，也可以充当关联数组（Map）。

在Java不断演化了许多年之后，研究这样一个问题会相当有趣：如果Java的设计者们能够从头再来，他们是否还会在Java语言中设计基本类型的低级数组。如果当初抛弃它们，Java也许就会成为真正的纯面向对象语言（不管人们如何宣传，Java仍旧不是纯面向对象语言，而原因正是这些低级的绊脚石）。最初有关效率的论点总是很吸引人，但是随着时间的推移，我们看到了与这种思想背道而驰，向着使用像容器这类高级构件的方向的演化。另外，如果容器能够像某些语言一样内置于语言的内核中，那么编译器就会得到更好的优化良机。

788

我们肯定还会使用数组，并且你在读代码的时候还会看到它，但是，容器几乎总是更好的选择。

练习25：(3) 用Java重写**PythonLists.py**。

789 ~ 790

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net处购买此文档。

⊖ 查看www.php.net。